NeuroVision: An Interactive Platform for Building, Training, and Inspecting Neural Networks with Measured Values in Synchronized 2D and 3D Views

K. Vishwa

Department of Computer Science and Engineering, Mahatma Gandhi Institute of Technology (A), Hyderabad – 500075, Telangana, India

vishwa9821@gmail.com · https://github.com/vishwa-10147/neurovision

***Abstract*—**Neural-network visualization tools tend to fall into two groups. Educational playgrounds let learners build and train toy networks but cannot open a real model. Model viewers show the graph of a real model but not what it computes. This paper presents **NeuroVision**, an open web platform that combines both experiences on one normalized graph contract. Users can design dense networks backed by seeded PyTorch models and train them in background jobs whose loss, accuracy and activations stream over a replayable WebSocket. They can import trained ONNX models, run them in an isolated, time-limited ONNX Runtime worker, and inspect per-operator outputs, CNN feature maps, convolution filters and attention heads. They can also stream metrics and activations from their own training scripts through a one-file, standard-library client. Three principles govern the design: every value shown is measured rather than animated, every payload is bounded, and every uploaded model is untrusted. Synchronized 2D (SVG) and 3D (instanced WebGL) renderers share a semantic selection state and degrade to aggregate views beyond explicit detail limits. The implementation passes 265 backend and 188 frontend tests. On a CPU laptop, educational networks reach 100% validation accuracy on two-moons and circles in about 2.5 s, and a PyTorch CNN exported to ONNX and run in the worker reproduces PyTorch's outputs to within 5.2×10^{-8}. Attention is recovered by pattern from a real `nn.MultiheadAttention` export, and warm inference inside the worker takes 0.06–0.21 ms on the sample models. Seven pretrained vision models, from MobileNetV3 to ViT-B/16 and a 3D video CNN, were exported, viewed and run on a real photo (ONNX matched PyTorch to 2.4×10^{-5}), and fine-tuned on a laptop GPU while the dashboard showed CPU, GPU, memory, ETA and training health live; this surfaced a BatchNorm configuration that cut MobileNetV3's test accuracy from 98.6% to 52.5%.

***Index Terms*—**neural-network visualization, explainable AI, ONNX, PyTorch, interactive machine-learning education, attention visualization, feature maps, WebGL, sandboxed inference.

I. Introduction

Understanding what a neural network computes is central both to learning deep learning and to debugging real models. Visual tools have become a standard aid. Interactive playgrounds let learners change the architecture of a small network and watch it train [16]. Explainers animate the computation of a fixed convolutional network in the browser [6]. Graph viewers such as Netron [4] and TensorBoard's graph visualizer [13] display the operators of production models, and dedicated tools visualize attention in transformers [10].

These tools rarely meet, however. A playground cannot open the model a student exported last week. A graph viewer can open it but cannot run it, so it shows *structure* without *behaviour*. Explainers are tied to one pre-built network. Moreover, visualizations often mix measured values with illustrative animation, which leaves a learner unable to tell what the model actually produced.

NeuroVision addresses this gap with one platform and one shared graph representation for two sources of models. Small dense networks are created and trained inside the application; real trained models are imported in the open ONNX format [2]. The same 2D and 3D renderers, inspector and value views serve both. A third path streams training telemetry from the user's own scripts, so real projects can be watched while they train on the user's own hardware.

The contributions of this work are:

1) A **normalized, versioned graph contract** shared by PyTorch-backed educational networks and imported ONNX models, with capabilities that state honestly what each model supports (Section IV).
2) A **measured-values-only learning engine**: seeded, explicitly instrumented PyTorch networks trained in cancellable background jobs that stream bounded, replayable events (Section V).
3) **Synchronized 2D and 3D renderers** driven by pure layout and scene-planning functions with explicit level-of-detail limits (Section VI).
4) A **safe model explorer**. It parses untrusted ONNX files without executing them, and runs them only in a separate process with wall-clock and memory limits. It returns bounded tensor views (feature maps, filters, attention heads), and labels inferred semantics such as attention as inferences (Section VII).

5) A **dependency-free live-run SDK** that streams metrics and activations from arbitrary PyTorch training loops (Section VIII).
6) An **evaluation** of correctness, learning behaviour, latency and safety bounds on the implemented system (Section IX), and applies the platform to seven pretrained vision models on a laptop GPU, from MobileNetV3 to ViT-B/16 and a 3D video CNN (Section X).

II. Related Work

A. Educational and explanatory visualizers

TensorFlow Playground [16] popularized direct manipulation of small feed-forward networks on 2D toy datasets, and NeuroVision's Training Lab adopts its moons, circles and spiral tasks. CNN Explainer [6] teaches convolution through an interactive view of one fixed network. Both are effective for teaching, but neither can load a user's model, and both are bound to their own model definitions.

B. Model graph viewers and experiment tracking

Netron [4] renders the operator graphs of ONNX and many other formats and is widely used for inspection, but it does not execute models. TensorBoard [13], [17] and hosted trackers such as Weights & Biases [8] chart training metrics and show dataflow graphs, but they require their own logging stacks and do not connect metrics to an inspectable, runnable graph view.

C. Interpreting internal representations

Visualizing feature maps and filters is a long-standing way to understand CNNs [20], and attention matrices are a common, if debated, window into transformers [10], [15]. Surveys of visual analytics for deep learning [9] call for tools that combine *what* a model is with *what* it computes. NeuroVision contributes an integrated, bounded and security-conscious implementation of these views for arbitrary ONNX models.

TABLE I
COMPARISON OF CAPABILITIES

Capability	Playground [16]	CNN Explainer [6]	Netron [4]	TensorBoard [13]	NeuroVision
Build and train a network	✓	–	–	via user code	✓
Open the user's trained model	–	–	✓	TF graphs	✓ (ONNX)
Run an imported model	–	fixed model	–	–	✓ (isolated)
Per-operator measured values	–	✓	–	–	✓
Feature maps / filters / attention	–	maps, filters	–	–	✓ (all three)
Live metrics from the user's script	–	–	–	✓	✓
Synchronized 2D and 3D views	–	–	–	–	✓

III. Design Principles

Four principles, recorded in the project's architecture document and its seven decision records (ADRs), shape every component:

- **Measured, not decorative.** Every value the UI presents as model behaviour comes from a computation the model actually performed: a forward pass, a training step or an ONNX Runtime run. Colours that encode only layer roles are labelled as such, and no signal animation is shown where no measured activations exist.
- **Bounded everywhere.** Tensors are referenced, never embedded, in graph JSON. Events, tensor slices, run results, maps and thumbnails all have hard size limits, so payloads stay small regardless of model size.
- **Untrusted by default.** Uploaded models are parsed in memory, never executed on upload, never deserialized via pickle, and never allowed to read files they reference. Execution is an explicit user action in a separate process.
- **Honest capabilities.** Each graph declares what it supports (graph, inference, training, activations, gradients, attention). Inferences such as *this softmax is attention* or *this axis is channels* are labelled as inferences in the UI.

IV. System Architecture

NeuroVision is a two-tier web application (Fig. 1). The **frontend** is a React and TypeScript single-page application with strict type checking. Shared client state is kept in Zustand, charts use Recharts, and the 3D view uses React Three Fiber on Three.js [3]. The **backend** is a FastAPI service on Python 3.12 that uses Pydantic for validation, PyTorch [11] for educational networks, and ONNX [2] and ONNX Runtime [1] for imported models. REST endpoints under `/api/v1` handle commands and queries, and a WebSocket carries training events. Every error uses one envelope, `{error: {code, message, details}}`.

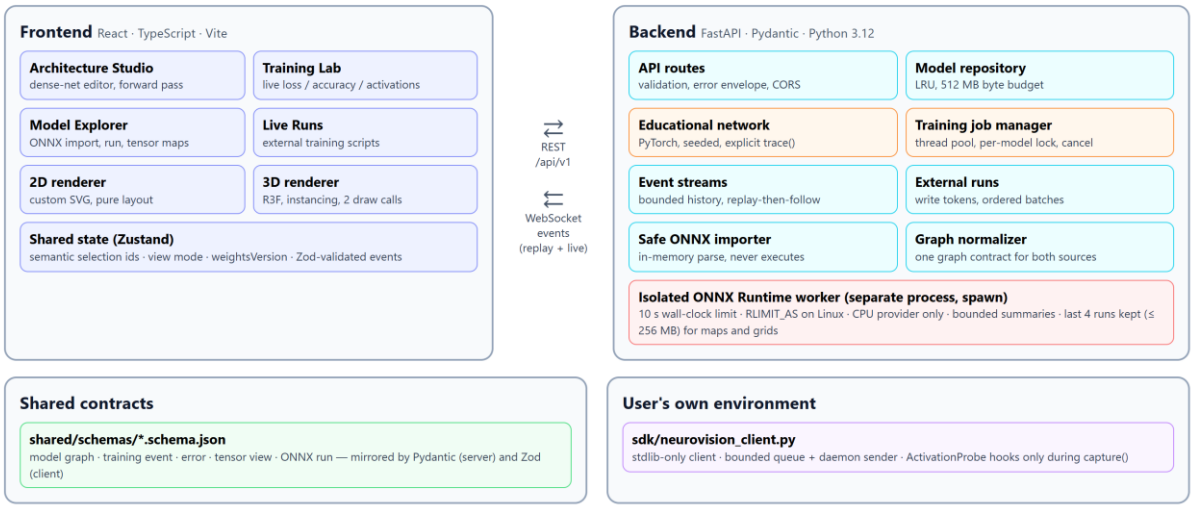


Fig. 1. System architecture. The frontend's four feature areas share renderers and a semantic selection store. The backend separates request handling from training (thread pool) and from model execution (a spawned ONNX Runtime worker). JSON Schemas in `shared/schemas/` are mirrored by Pydantic on the server and by Zod on the client.

A. Normalized graph contract

The integration boundary is a versioned JSON Schema (`model-graph.schema.json`). A graph has a model header (id, name, source format, capabilities), *nodes* (id, operator type, input and output tensor names and shapes, attributes, parameter summary) and *edges* (source, target, tensor name). Several rules keep it honest:

- Node ids are stable, and layout coordinates are never stored in the graph.
- Tensors are referenced by id rather than embedded.
- Unknown operators are preserved and flagged as `unsupported`.
- Unknown or dynamic dimensions become `null` rather than invented numbers.

Educational graphs map one dense layer to one `Linear` node. ONNX graphs map graph inputs to `Input` nodes, operators to nodes with generated ids, and graph outputs to `Output` nodes. Weights (initializers) are listed on their consumers rather than drawn as nodes, because they are parameters, not steps of computation.

B. Contracts kept in sync

Each contract exists three times: as a JSON Schema with examples, as Pydantic models, and as Zod schemas in the client. Backend tests validate the examples against the schemas and the API responses against the Pydantic models. The client validates every WebSocket event with Zod before it reaches the store, so a contract drift fails loudly instead of rendering wrong values.

V. Learning Engine

A. Instrumented educational networks

The Architecture Studio builds a dense network with 1–16 inputs, up to six hidden layers of 1–32 units (ReLU, sigmoid or tanh) and 1–10 outputs. The backend and the frontend both enforce these limits. Instead of opaque forward hooks, `EducationalNetwork.trace()` records each layer's pre-activation `W·x + b` and post-activation value *as the forward pass computes them*; training calls `forward()`, which returns logits only. Weight initialization is seeded inside `torch.random.fork_rng`, and the datasets and batch order use their own generators. The same settings therefore reproduce a run exactly, which the tests assert and Section IX confirms.

B. Background training jobs

Training never runs on the ASGI event loop. Jobs execute in a thread pool (two workers by default). Further requests are rejected with HTTP 429 rather than queued, so the server never accumulates unbounded work. Every optimizer step, validation pass, forward request and tensor read holds a per-model lock, so a reader never observes a half-applied update. Cancellation is a `threading.Event` checked every batch. A non-finite loss stops the job with an actionable message instead of streaming NaNs. One output unit trains a binary classifier with BCE-with-logits; more outputs use cross-entropy. The optimizer is Adam, or SGD with momentum 0.9.

C. Replayable event streaming

Each job keeps a bounded event history (at most *epochs* + *2* small events) and delivers to WebSocket subscribers via `loop.call_soon_threadsafe`. On subscription the server snapshots the history and registers the subscriber under one lock, so no event is lost or duplicated between replay and live delivery. A late or reconnecting client therefore sees the complete run.

Step events carry four metrics plus at most 8 layers × 256 activation values for one fixed validation example, never weights. The client batches store updates to at most 20 per second and reconnects up to three times, deduplicating by epoch. When a job ends it increments a `weightsVersion`. Forward results and tensor views are keyed by that version, so stale values are hidden rather than shown as current. Fig. 2 shows the Training Lab during a run.

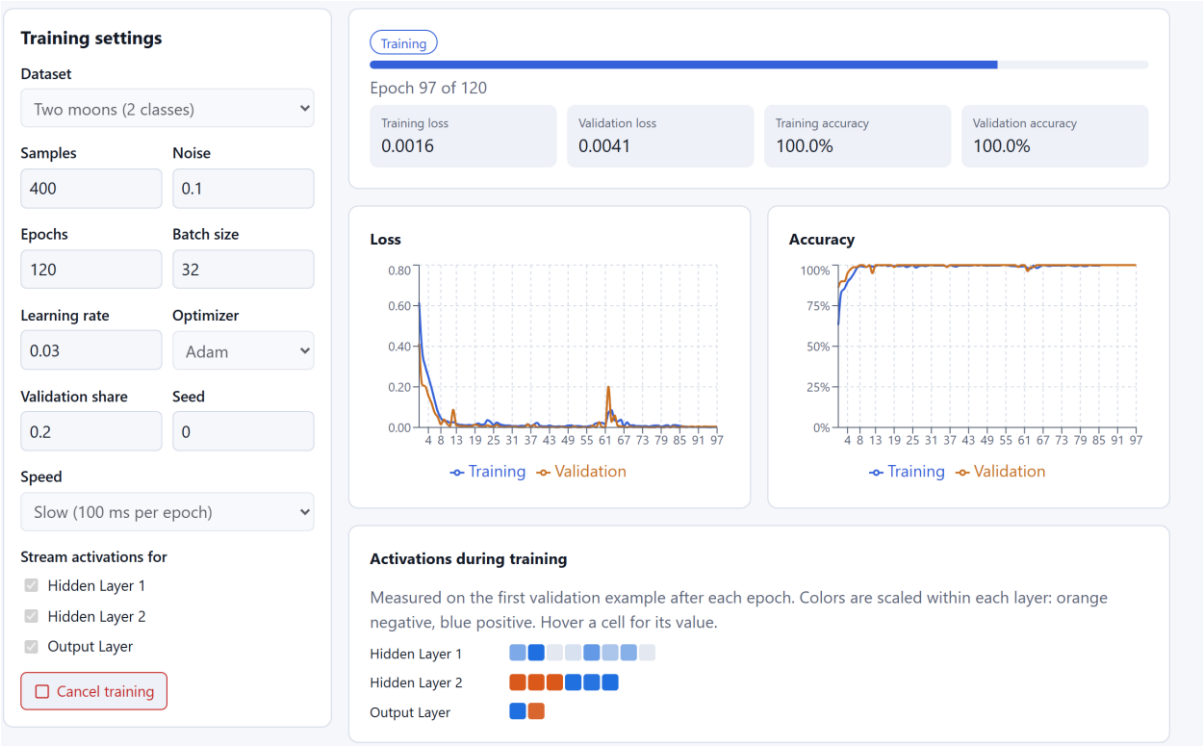


Fig. 2. Training Lab on two moons (2 → 8 → 6 → 2, Adam, learning rate 0.03) at epoch 97 of 120. Loss and accuracy curves and per-layer activations of the first validation example are streamed live over the WebSocket; validation accuracy has reached 100%.

VI. Synchronized 2D and 3D Visualization

Both renderers consume the same graph and the same selection state. Selection is stored as *semantic* ids (a layer, a unit index, or a connection with optional unit indices), never as references to SVG or WebGL objects. Clicking in either view, or in the layer buttons provided for keyboard users, updates the inspector, and switching views preserves the selection (Fig. 3).

A. 2D renderer

The neuron-level view is a custom SVG renderer. React Flow was considered, but nothing yet requires free-form editing. Layout is a pure function of the graph (`computeLayout`) and is never persisted. Above 1,500 unit-to-unit connections, each layer pair is drawn as one band, which bounds the DOM to about 1,500 lines plus at most 218 units. After a forward pass, units are shaded by their measured values on a diverging scale scaled within each layer: orange for negative values, blue for positive.

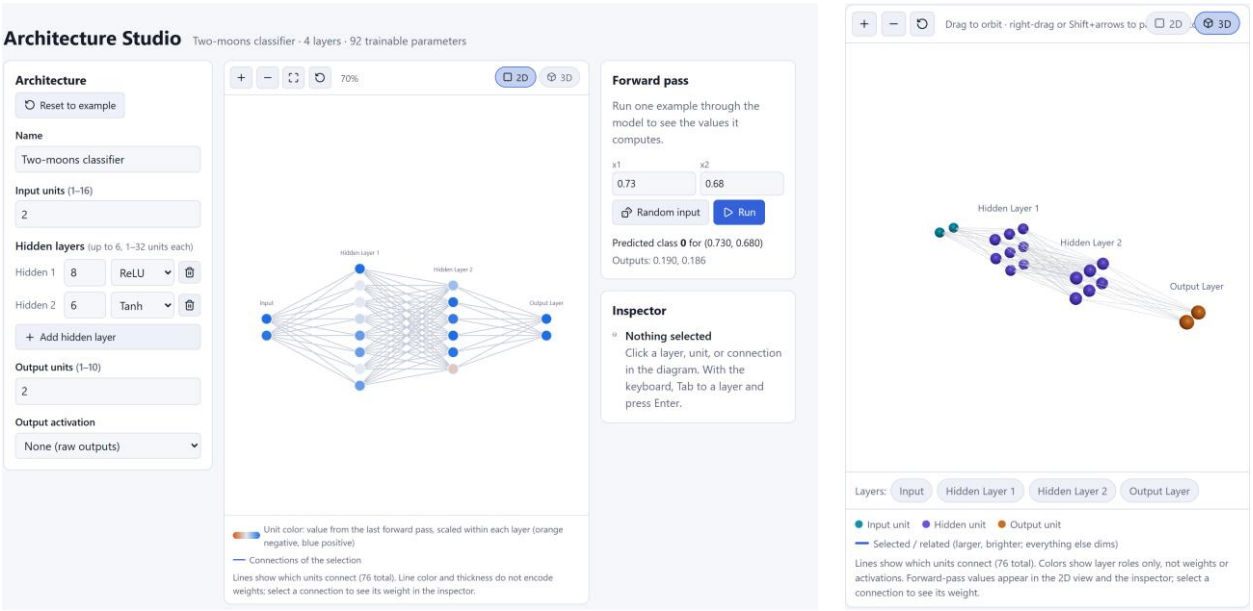


Fig. 3. Architecture Studio. Left: 2D view after a forward pass on input (0.73, 0.68); units are shaded by their real activations, and the model predicts class 0 with outputs (0.190, 0.186). Right: the same network in the 3D view, where each layer is a near-square grid and colours encode layer roles only, as the legend states.

B. 3D renderer

Planning is separated from drawing. `planScene(graph)` is a pure, unit-tested function that places each layer as a near-square grid and spaces the layers along the x axis. It also produces connection pairs, layer links, detail flags and bounds. All units are drawn as one `InstancedMesh` and all connections as one `LineSegments` with vertex colours, which makes two draw calls regardless of network size. Rendering is on demand.

Above 2,048 units, layers become solid blocks; above 8,000 connections, layer pairs become single links; and unknown layer sizes always render as blocks. The legend reports which limit applied. The editor's largest network ($16 \to 6 \times 32 \to 10$: 218 units, 5,952 connections) stays at full detail. Two further details complete the view:

- **Camera framing.** The camera distance is binary-searched until every corner of the scene's bounding box, labels included, projects inside the frustum for the canvas's real aspect ratio.
- **Lazy loading.** Three.js and R3F (about 945 kB, 252 kB gzipped) load only on the first switch to 3D, and an error boundary falls back to the 2D view if WebGL fails.

VII. Model Explorer: Safe Import, Isolated Inference, and Tensor Views

A. Non-executing import

An upload is read as the raw request body, with its size checked from `Content-Length` and again while streaming (64 MB default). It is parsed with `onnx.load_model_from_string`, which never follows external-data paths; a test fails on any `open` or `stat` call during import. When a model references external data, the ONNX checker is skipped because it could probe the filesystem.

Empty, unparsable or graph-less files, and graphs with more than 5,000 operators, are rejected with client-safe messages, and every string copied from the file is truncated. The import report lists operator, edge and initializer counts, opsets, unsupported operators and warnings. It also lists *inference blockers*, such as non-standard operators, external weights or unsupported input types. Imported models share an LRU repository, with a 512 MB budget, with educational models.

B. Process-isolated inference

The API process never imports ONNX Runtime; a test asserts this. Runs execute in one long-lived worker process started with `spawn`, which caches up to four sessions and communicates over a pipe. Each run has a 10 s wall-clock limit: on timeout the worker is killed, the request returns 504, and a fresh worker starts on the next run.

The parent polls the worker every 50 ms, so a crash returns 500 at once. On Linux and macOS the worker's address space is capped with `RLIMIT_AS` (2 GiB by default). Only the CPU execution provider is enabled, and only

one run proceeds at a time. Inputs must match the declared names, ranks and fixed dimensions, with at most 10^6 values in total. They are either typed by the user or generated deterministically from a seed.

For graphs up to 200 operators, the parent builds, without executing it, a variant of the model that exposes every operator output. The worker then returns each tensor as a bounded summary: shape, finite min, max, mean and standard deviation, NaN/∞ count, and at most 4,096 values. Reported latency is execution time measured inside the worker; worker start-up and model loading are reported separately as notes (Fig. 4).

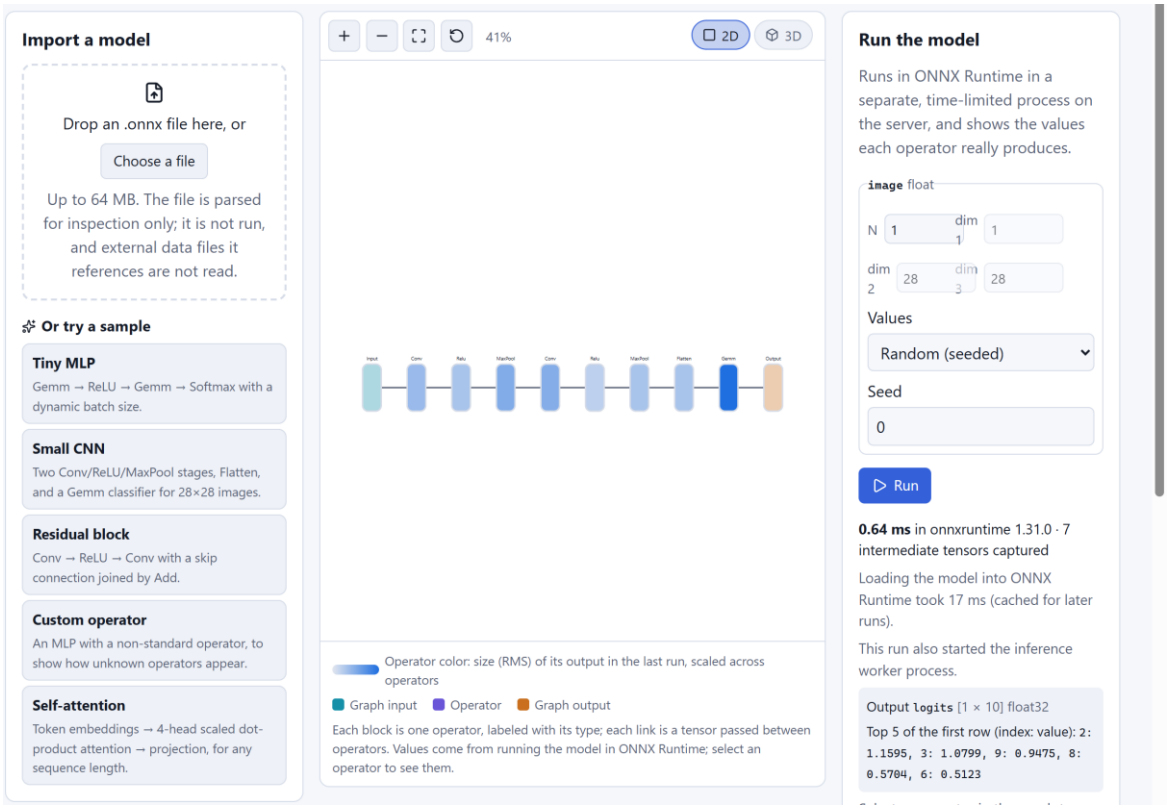


Fig. 4. Model Explorer after running the sample CNN. Each block is one ONNX operator, coloured by the RMS of its output in the last run. The run panel reports 0.64 ms of execution inside ONNX Runtime 1.31.0 and seven captured intermediate tensors, and notes separately that this run also started the worker process.

C. Bounded tensor views

Large activations, such as a [1, 64, 112, 112] tensor with 800,000 values, are never sent whole. After a run the *worker*, not the API, keeps every tensor of the last four runs within a 256 MB budget, and a stale run returns HTTP 410 with advice to run again. Two endpoints read from these kept tensors:

- `maps` returns one 2D slice of the last two dimensions at given leading indices. Slices over 65,536 cells are shrunk by block averaging with a square stride $s = \lceil\sqrt{(\text{rows}\cdot\text{cols} / 65{,}536)}\rceil$, which the response reports; NaN and ∞ values are excluded from the averages and counted.
- `grids` returns thumbnails along the third-to-last dimension, such as channels or heads: at most 256 items of at most 24 × 24 cells, sharing one colour range.

The downsampling is pure NumPy and tested against hand-computed block means. Convolution filters are drawn from the bounded weight endpoint, with at most 16 filters × 8 input channels shown (Fig. 5).



Fig. 5. Left: feature maps of the first ReLU of the sample CNN (8 channels × 28 × 28) for a seeded input, with channel 3 selected. Dimension roles are labelled as an NCHW assumption, and colours share one scale across all maps. Right: the 8 first-layer 3 × 3 convolution filters on a diverging scale with full colour at ±0.233.

D. Attention detected by pattern

ONNX does not mark which tensor is attention. NeuroVision therefore reports a candidate whenever it finds a `Softmax` over the last axis of a tensor of rank ≥ 3 whose last two dimensions are equal, either numerically or by symbolic name. For rank 4 it guesses the layout [batch, heads, query, key]. Wherever such a candidate is shown, the UI says *detected by pattern*. Heads use a fixed 0–1 colour scale so that they are comparable, and the viewer notes that each row of a softmax over keys sums to 1 (Fig. 6). Fused `com.microsoft` attention operators and softmaxes over other axes are not detected, but they can still be viewed as plain tensor maps.



Fig. 6. Attention weights of the sample self-attention model for a 12-token seeded input: all four heads as thumbnails, with head 2 enlarged as a query × key map. The header states that attention was detected by pattern.

VIII. Live Runs from Users' Own Training Scripts

To watch real projects rather than only built-in networks, NeuroVision accepts *pushed* telemetry and never runs user code. A script creates a run with `POST /runs` and receives a random write token (`secrets.token_urlsafe(32)`). The server stores only the token's SHA-256 digest and compares tokens with `hmac.compare_digest`.

Batches of up to 32 steps are posted to `/runs/{id}/events` and are validated in full before any step is published. Step numbers must increase and epochs must not decrease. Each step carries at most 16 finite metrics and 8 layers × 256 activation values. A run accepts at most 10,000 steps, at most 8 runs can be active at once, and a run that is idle for 5 minutes is marked failed. External runs reuse the replayable event stream of Section V-C, so the browser follows them on the same WebSocket.

The client, `sdk/neurovision_client.py`, is a single file that uses only the standard library. `log()` enqueues into a bounded deque and returns at once, and a daemon thread sends batches about every 0.5 s, dropping the oldest steps under backpressure. The client never raises into the training loop. `ActivationProbe` attaches PyTorch forward hooks only during `capture()`, so ordinary training forward passes pay nothing. The Live Runs view charts every metric the script chooses and pairs `x` with `val_x` (Fig. 7).

Every logged step can also carry system statistics measured by the script. The client reports elapsed time, the time since the previous step, throughput when the script passes the number of samples, and an ETA from the rate of progress over the last 20 logged steps; a fractional progress argument keeps the ETA right when a script logs several times per epoch. When psutil and NVML (nvidia-ml-py) are installed, a SystemMonitor adds machine CPU load and RAM, the script's own CPU and resident memory, PyTorch's CUDA allocator, and for each GPU its utilization, memory, temperature and power. The server validates these bounds like any other field, and two rules turn them into alerts: a GPU at most 20% busy for five consecutive steps (the input pipeline is likely the bottleneck) and GPU memory at least 95% full.

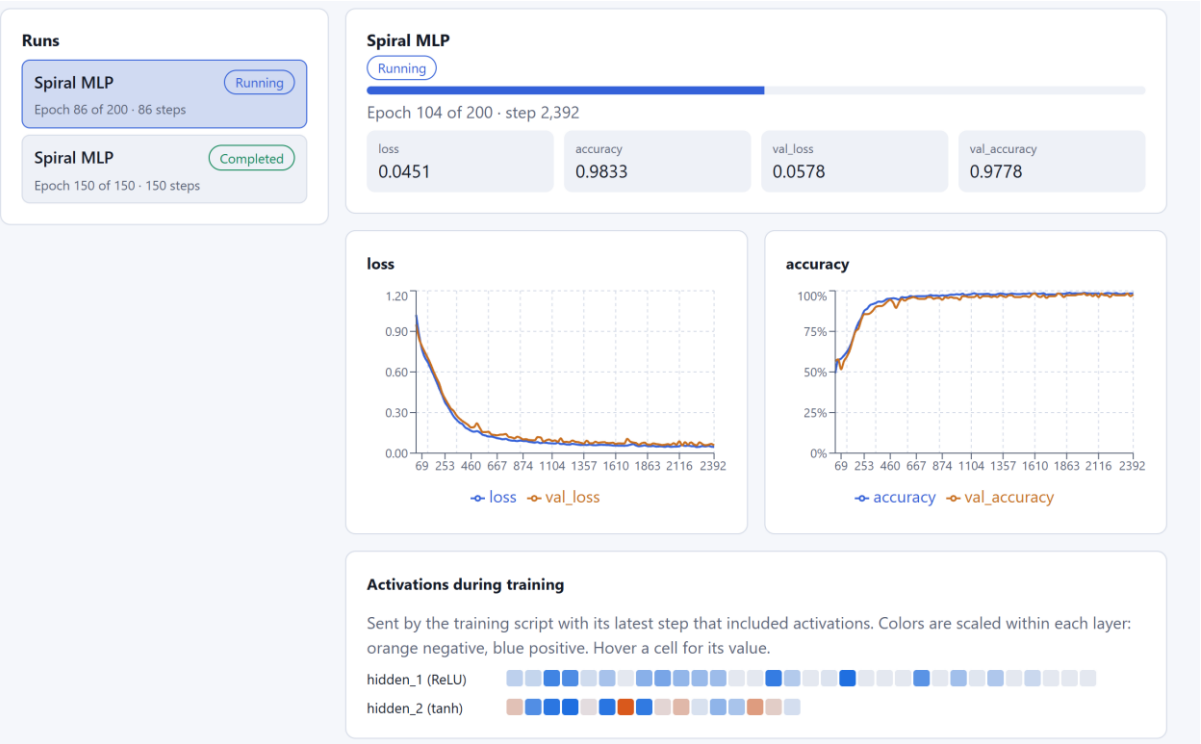


Fig. 7. Live Runs streaming the SDK's spiral example: a PyTorch MLP training in a separate process on the same machine. Metrics are plotted against the script's own step numbers, and post-activation values of two hidden layers arrive with the latest step.

IX. Evaluation

All measurements were taken on a laptop with an Intel Core i9-13980HX, 15.6 GB of RAM and Windows 11. The software was Python 3.12.10, PyTorch 2.14.1 (CPU build), ONNX 1.23.2, ONNX Runtime 1.31.0, FastAPI 0.147.0 and Node.js 24. API latencies were measured in process through FastAPI's test client, so they include validation and serialization but not network transfer. Section X additionally used an NVIDIA GeForce RTX 4070 Laptop GPU (8 GB) with PyTorch 2.14.1 built for CUDA 13.0.

A. Test suites and build

The backend suite (pytest) passes **265 tests in 13.7 s**. The tests cover schemas, the error envelope, numerical forward passes and training, determinism, cancellation, ONNX import (malformed, oversized and external-data cases), isolation of the inference worker, timeouts, tensor views, external runs, training-health rules and system telemetry. The frontend suite (Vitest with Testing Library) passes **188 tests in 22 files** in 7.1 s. Strict `tsc -b` type checking is clean, and the production build completes in under 1 s; the 3D view is split into its own lazy-loaded chunk.

B. Numerical correctness

A small PyTorch CNN (Conv–ReLU–MaxPool–Conv–ReLU–AdaptiveAvgPool–Linear) was exported to ONNX, uploaded through the public import endpoint, and run in the isolated worker on the same $1 \times 3 \times 32 \times 32$ input. The maximum absolute difference from PyTorch's own logits was $\mathbf{5.2 \times 10^{-8}}$, and all seven intermediate tensors were captured. A `torch.nn.MultiheadAttention` export (embedding 32, 4 heads, 10 tokens; 54 operators) was imported, and its attention softmax was **detected automatically** as a [4, 10, 10] tensor. The repository's own tests additionally check per-head attention weights against PyTorch to 1×10^{-5}.

C. Learning behaviour and determinism

Table II reports training through the public API with default settings: Adam, learning rate 0.03, batch size 32, noise 0.1 and a 20% validation split. The spiral run used 900 samples and a learning rate of 0.01. Running the same configuration twice gave identical final loss and validation accuracy, confirming the seeded determinism of Section V-A.

TABLE II
TRAINING RESULTS THROUGH THE API (CPU, FULL SPEED)

Dataset	Network	Epochs	Wall time	Loss: first → last	Train acc.	Val. acc.
Two moons	2 → 16 → 16 → 1	100	2.50 s	0.581 → 0.0001	100.0%	100.0%
Circles	2 → 16 → 16 → 1	100	2.38 s	0.669 → 0.003	100.0%	100.0%
Spiral (3 arms)	2 → 32 → 32 → 3	300	16.35 s	0.806 → 0.126	94.3%	92.8%

D. Latency

Table III summarizes the latencies. Creating an educational model and returning its graph takes about 3.3 ms, even for the largest allowed network (6,154 parameters, 5,952 connections, a 3.8 kB graph). A forward pass through that network with all activations returned takes 2.2 ms for one row and 4.5 ms for 64 rows. Importing a sample ONNX model takes 4–6 ms; the first import of the session took 48 ms.

For inference, the first run of the session paid a one-time worker start-up cost of 924 ms. Warm execution inside the worker then took 0.06–0.21 ms, and the full request, including the capture and summary of every intermediate tensor, took 3–23 ms. Bounded thumbnail grids of a CNN feature map and of the attention heads took about 4–5 ms.

TABLE III
MEASURED LATENCIES (MEDIAN OF REPEATED REQUESTS)

Operation	Model	Median
Create educational model + graph	2 → 8 → 6 → 2 / 16 → 6×32 → 10	3.26 / 3.30 ms
Forward pass, all activations	16 → 6×32 → 10, batch 1 / 64	2.20 / 4.51 ms
ONNX import (sample)	CNN / residual / attention	4.1 / 3.7 / 6.2 ms
First run (starts worker)	Tiny MLP	924 ms
Run request, warm (all intermediates)	MLP / CNN / residual / attention	2.9 / 19.3 / 22.8 / 11.2 ms
Execution inside worker	MLP / CNN / residual / attention	0.06 / 0.17 / 0.19 / 0.21 ms
Thumbnail grid (bounded)	CNN conv1 (8 ch.) / attention (4 heads)	4.73 / 4.24 ms

E. Safety and bounds

The import endpoint rejected an empty body and a non-ONNX byte string with `422`, and an architecture with 99 inputs was rejected with `422` naming the offending field. Of the five built-in samples, four were marked runnable. The custom-operator sample was correctly marked *not runnable*, with the blocker *non-standard operator com.example::Scale*, while remaining fully inspectable. Table IV lists the enforced limits.

TABLE IV
ENFORCED LIMITS

Resource	Limit
Educational architecture	≤ 16 inputs, ≤ 6 hidden layers × 32 units, ≤ 10 outputs
Concurrent training jobs / external runs	2 (then HTTP 429) / 8 active, 32 finished kept
Training event	4 metrics + ≤ 8 layers × 256 values (jobs); ≤ 16 metrics (runs)
Upload size / operators	64 MB / 5,000 operators
Inference	10 s wall clock; 2 GiB address space (Linux/macOS); ≤ 10^6 input values
Run results	≤ 4,096 values per tensor, every operator captured up to 200 operators
Kept run tensors	last 4 runs, ≤ 256 MB, held in the worker process
Maps / grids	≤ 65,536 cells (block-averaged) / ≤ 256 items × 24 × 24
Model repository	512 MB, least-recently-used eviction

X. Heavy Vision Models: Measurements, Views and Live GPU Telemetry

To check that the platform holds up beyond toy networks, seven pretrained torchvision models were measured and taken through every view: MobileNetV3-Large [12] and EfficientNet-B0, ResNet-18 and ResNet-50 [18], ConvNeXt-Tiny [5], the Vision Transformer ViT-B/16 [7], and the 3D video CNN R3D-18 [14] (input 16 frames of 112 × 112). The scripts and raw results are in the repository's bench/ folder. Throughout the GPU measurements the card drew only 52–55 W at 96–100% utilization and 69–73 °C, so it was power-limited; the GPU figures are a lower bound for this hardware.

A. Inference cost and load

Table V lists size, cost and speed. Batch-1 latency on the GPU is dominated by kernel-launch overhead: the 0.22-GMAC MobileNetV3 (6.1 ms) is slower than the 1.8-GMAC ResNet-18 (2.8 ms), while throughput at batch 64 with fp16 autocast follows the arithmetic (3,531 vs 2,694 images/s). During every throughput test the GPU was the bottleneck (96–100% busy, CPU 10–27%). On the CPU, ONNX Runtime ran MobileNet, EfficientNet and the ResNets 5–14× faster than eager PyTorch, but ConvNeXt and ViT only 1.2–1.3× faster and the 3D CNN slower. R3D-18 costs 40.7 GMACs per clip, about ten times ResNet-50, because every convolution also slides over time.

TABLE V
PRETRAINED VISION MODELS ON AN RTX 4070 LAPTOP GPU (BATCH-1 LATENCY FP32; THROUGHPUT FP16)

Model	Params / GMACs	GPU b1	GPU throughput	ORT CPU b1	Max |ONNX − PyTorch|
MobileNetV3-L	5.5 M / 0.22	6.1 ms	3,531 /s	1.8 ms	5.8×10^{-6}
EfficientNet-B0	5.3 M / 0.39	9.1 ms	1,955 /s	4.8 ms	9.5×10^{-6}
ResNet-18	11.7 M / 1.81	2.8 ms	2,694 /s	5.3 ms	6.7×10^{-6}
ResNet-50	25.6 M / 4.09	6.5 ms	869 /s	12.0 ms	2.4×10^{-5}
ConvNeXt-Tiny	28.6 M / 4.46	4.5 ms	752 /s	34.7 ms	5.7×10^{-6}
ViT-B/16	86.6 M / 11.3	9.6 ms	401 /s (b32)	108 ms	1.8×10^{-5}
R3D-18 (video)	33.4 M / 40.7	10.2 ms	187 clips/s (b16)	300 ms	4.5×10^{-6}

B. Exporting, importing and viewing

All seven ONNX exports reproduced PyTorch's logits to within 2.4×10^{-5} with the same top-1 class. ResNet-50 (97 MB, 122 operators) imported in 1.0–1.8 s and ViT-B/16 (331 MB, 1,016 operators) in 3.4–5.6 s. Fed the same photo of a pagoda through the new photo input, which crops and normalizes the image in the browser, both reported ImageNet class 698 (palace) first, as PyTorch did. A photo of a dahlia, which has no ImageNet class, drew low-confidence guesses such as daisy and honeycomb: a closed label set is a limit of the models, not of the tool.

Heavy models exposed weaknesses in the first design, and four changes followed. As a single row, ResNet-50 fitted to the screen was a line at 5% zoom; chains longer than 32 operators now wrap into rows that run alternately left to right and right to left (laid out in depth in 3D, Fig. 8), sized to about 16:9. ViT's constants and shape arithmetic all fell into the first column and blocked wrapping; operators not fed by a graph input are now placed just before their first consumer. Above 200 operators intermediates are not captured automatically, so the inspector and the attention list now offer to capture an operator and run again. Finally, a shared color scale left most ResNet channels and every ViT head nearly blank (activations reach 34 while the mean is 0.8; 197 attention weights per row average 0.005); an option to scale each map on its own makes their structure visible (Fig. 9). Five-dimensional video activations are read as [batch, channels, frames, height, width], one map per frame.

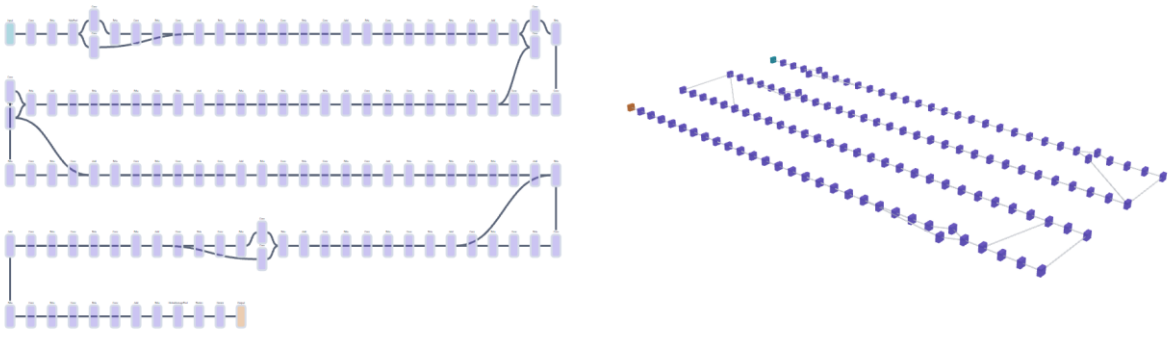


Fig. 8. ResNet-50 (122 operators) imported from ONNX. Left: 2D view; long chains wrap into rows that alternate direction. Right: the same rows laid out in depth in the 3D view.



Fig. 9. Left: ResNet-50 layer1.0 ReLU outputs (256 channels, 56 × 56) for the pagoda photo, each map on its own scale; the tower and the shoreline are visible. Right: ViT-B/16 encoder layer 0 attention (12 heads, 197 × 197 tokens), captured on request and scaled per head; heads 2, 5, 8 and 10 show clear structure.

C. Live GPU fine-tuning

Each classifier head was replaced with ten outputs and fine-tuned on the 1,797 handwritten digits that ship with scikit-learn (8 × 8, upsampled on the GPU to 224 × 224; 1,437 training and 360 test images), with AdamW, a one-cycle schedule, fp16 autocast and GradScaler. Every four batches the script logged loss and accuracy, HealthMonitor statistics and system figures to Live Runs (Fig. 10). Table VI summarizes the runs.

TABLE VI
FINE-TUNING ON DIGITS WITH LIVE TELEMETRY (RTX 4070 LAPTOP GPU)

Model	Epochs	Train time	Images/s	Test accuracy	Peak GPU memory
ResNet-18	5	28.8 s	249	98.9%	1.6 GB
ResNet-50	6	64.9 s	133	99.4%	3.3 GB
ConvNeXt-Tiny	5	71.3 s	101	99.4%	5.3 GB
ViT-B/16 (batch 32)	5	82.3 s	87	99.7%	4.1 GB
EfficientNet-B0	5	93.7 s	77	98.6%	2.9 GB
MobileNetV3-L	5	79.1 s	91	52.5%	1.5 GB
MobileNetV3-L, BN momentum 0.1	5	67.4 s	107	98.6%	1.5 GB

MobileNetV3 reached 99% training accuracy but only 52.5% on the test set, and Live Runs showed the gap from the first epochs. torchvision builds this model with a BatchNorm [19] momentum of 0.01, so in about 115 steps the running statistics barely moved from ImageNet's towards the new images, and evaluation, which uses them, failed. With momentum 0.1 the same run reached 98.6%. It is the only model here built with 0.01, and the only one that failed. A health alert also fired for a squeeze-and-excitation ReLU in that model that was dead on the whole probe

batch. A first ViT run raised a false non-finite-gradient alert from GradScaler's deliberate overflow steps; the script now skips health collection on steps the scaler rejected, and the alert's advice explains the case.

Monitoring is not free: collecting per-layer health every four batches cut throughput by about 30% for ResNet-18 (249 vs 359 images/s without it) and 20% for MobileNetV3, so long runs should collect less often. During training the GPU was mostly 90–100% busy, dipping during the per-epoch test pass and health collection, at 49–55 W and 67–72 °C, while the CPU stayed at 15–30%.

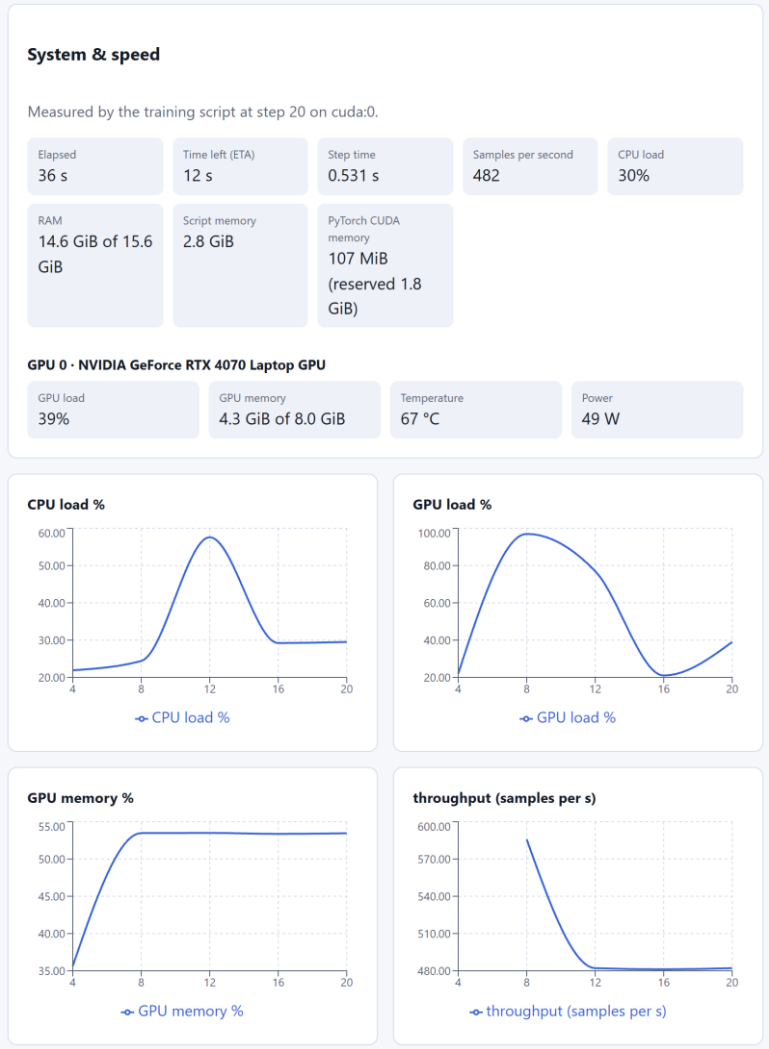


Fig. 10. System & speed panel during a MobileNetV3 fine-tune: elapsed time, ETA, step time, throughput, CPU, RAM, script and CUDA memory, and per-GPU load, memory, temperature and power, charted over logged steps.

D. Where this is useful

The same measurements answer practical questions. For deployment, they show which models fit a budget: MobileNetV3 or ResNet-18 run in 2–5 ms per image on a laptop CPU through ONNX Runtime, while ViT-B/16 and 3D CNNs need a GPU. For transfer learning, live train-versus-test curves, update ratios and dead-unit alerts catch configuration errors such as the BatchNorm momentum above early rather than after a run. For performance, GPU-idle and memory alerts, throughput and ETA show whether the data pipeline or the model is the bottleneck and whether a change helped. For explanation and teaching, feature maps on a real photo and per-head attention show what a CNN or a transformer responds to, and importing an export and comparing its top classes verifies that a deployed model is the trained one. The 5D view extends this to video and volumetric (for example CT or MRI) models.

XI. Limitations and Future Work

NeuroVision is in early development, and several limits are deliberate:

- **In-memory state.** Models, jobs and runs are held per process and lost on restart. Multi-user deployment needs a shared job store, a worker pool and authentication; today write tokens protect only external-run writes.

- **Platform limits on isolation.** Windows has no `RLIMIT_AS` equivalent, so there the process boundary and the time limit are the only protections against a hostile model. Untrusted models should be run on Linux or in a memory-limited container.
- **Inference of semantics.** Dimension roles assume NCHW, so channels-last models show rows × channels maps, and the viewer states this assumption. Fused attention operators are not detected. ONNX carries no tokenizer, so attention axes are labelled by position only.
- **Scale of graphs.** Long operator chains wrap into rows and constants are placed beside their consumers, so a 1,016-operator ViT fits one screen, but operators are not yet grouped by module, subgraphs are not expanded, and above 200 operators intermediate values are kept only for operators the user asks for.
- **Training statistics.** Gradients shown in the inspector are a snapshot from the last batch of each epoch, not an epoch average.
- **Self-reported telemetry.** System figures come from the training script's own process; NVML utilization is the share of time a kernel ran, not how much of the GPU it used, and GPU memory includes other processes. Per-layer health collection costs speed (20–30% when collected every four batches).

Planned work includes a guided Learning Mode for dense networks, CNNs and transformers; saved projects; drill-down and module grouping for large graphs; weight-coloured connections; and a pooled, containerized inference service.

XII. Conclusion

NeuroVision shows that one platform can teach with small trainable networks and also inspect real trained models, without sacrificing honesty or safety. A single normalized graph contract feeds synchronized 2D and 3D views. Every displayed value is measured, every payload is bounded, and uploaded models are parsed without execution and run only in an isolated, time-limited worker.

The implementation reproduces PyTorch's outputs to 5.2×10^{-8} through ONNX, recovers attention heads from a standard PyTorch export, trains toy tasks to 100% validation accuracy in seconds on a CPU, and answers typical requests in milliseconds. With real pretrained models it imports a 331 MB ViT-B/16 in about 4 s, shows its feature maps and attention on a real photo, and streams GPU fine-tuning with live system load and ETA, which made a model-specific BatchNorm problem visible within the first epochs. The source code, design records and documentation are available at https://github.com/vishwa-10147/neurovision.

References

[1] ONNX Runtime Developers, "ONNX Runtime: Cross-platform, high performance ML inferencing and training accelerator," software, v1.31, 2026. [Online]. Available: https://onnxruntime.ai

[2] ONNX Project Contributors, "Open Neural Network Exchange (ONNX)," software and specification, v1.23, 2026. [Online]. Available: https://onnx.ai

[3] R. Cabello et al., "Three.js: JavaScript 3D library," software, r186, 2026, https://threejs.org; Poimandres, "React Three Fiber," software, v9.8, 2026, https://github.com/pmndrs/react-three-fiber.

[4] L. Roeder, "Netron: Visualizer for neural network, deep learning and machine learning models," software, accessed Oct. 2026. [Online]. Available: https://github.com/lutzroeder/netron

[5] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, "A ConvNet for the 2020s," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), 2022, pp. 11976–11986.

[6] Z. J. Wang, R. Turko, O. Shaikh, H. Park, N. Das, F. Hohman, M. Kahng, and D. H. Chau, "CNN Explainer: Learning convolutional neural networks with interactive visualization," IEEE Trans. Vis. Comput. Graphics, vol. 27, no. 2, pp. 1396–1406, 2021.

[7] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," in Proc. Int. Conf. Learning Representations (ICLR), 2021.

[8] L. Biewald, "Experiment tracking with Weights and Biases," software, https://wandb.ai, 2020.

[9] F. Hohman, M. Kahng, R. Pienta, and D. H. Chau, "Visual analytics in deep learning: An interrogative survey for the next frontiers," IEEE Trans. Vis. Comput. Graphics, vol. 25, no. 8, pp. 2674–2693, 2019.

[10] J. Vig, "A multiscale visualization of attention in the Transformer model," in Proc. 57th Annu. Meeting Assoc. Comput. Linguistics: System Demonstrations, 2019, pp. 37–42.

[11] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in Advances in Neural Information Processing Systems (NeurIPS), 2019, pp. 8024–8035.

[12] A. Howard, M. Sandler, G. Chu, L.-C. Chen, B. Chen, M. Tan, W. Wang, Y. Zhu, R. Pang, V. Vasudevan, Q. V. Le, and H. Adam, "Searching for MobileNetV3," in Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV), 2019, pp. 1314–1324.

[13] K. Wongsuphasawat, D. Smilkov, J. Wexler, J. Wilson, D. Mané, D. Fritz, D. Krishnan, F. B. Viégas, and M. Wattenberg, "Visualizing dataflow graphs of deep learning models in TensorFlow," IEEE Trans. Vis. Comput. Graphics, vol. 24, no. 1, pp. 1–12, 2018.

[14] D. Tran, H. Wang, L. Torresani, J. Ray, Y. LeCun, and M. Paluri, "A closer look at spatiotemporal convolutions for action recognition," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), 2018, pp. 6450–6459.

[15] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in Advances in Neural Information Processing Systems (NeurIPS), 2017, pp. 5998–6008.

[16] D. Smilkov, S. Carter, D. Sculley, F. B. Viégas, and M. Wattenberg, "Direct-manipulation visualization of deep networks," in ICML Workshop on Visualization for Deep Learning, 2016.

[17] M. Abadi et al., "TensorFlow: A system for large-scale machine learning," in Proc. 12th USENIX Symp. Operating Systems Design and Implementation (OSDI), 2016, pp. 265–283.

[18] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2016, pp. 770–778.

[19] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in Proc. 32nd Int. Conf. Machine Learning (ICML), 2015, pp. 448–456.

[20] M. D. Zeiler and R. Fergus, "Visualizing and understanding convolutional networks," in Proc. European Conf. Computer Vision (ECCV), 2014, pp. 818–833.